UNIVERSIDAD DE GRANADA

ETSIIT INFORMÁTICA Y TELECOMUNICACIÓN

UNIVERSIDAD DE GRANADA

Departamento de Ciencias de la Computación e Inteligencia Artificial

Metaheurísticas

Guión de Prácticas

Práctica 1: Técnicas de Búsqueda de Poblaciones para el Problema de Asignación Con Restricciones (PAR)

Curso 2025-26

Tercero en Grado en Ingeniería Informática

1 OBJETIVOS

El objetivo de esta práctica es estudiar el funcionamiento de las *Técnicas de Búsqueda Local y de los Algoritmos Greedy* en la resolución del Problema de Asignación Con Restricciones (PAR) descrito en las transparencias del Seminario 2. Para ello, se requerirá que el estudiante adapte los siguientes algoritmos a dicho problema:

- Algoritmo Greedy básico.
- Algoritmo Aleatorio.
- Algoritmos de Búsqueda Local (BL).

El estudiante deberá comparar los resultados obtenidos con las estimaciones existentes para el valor de los óptimos de una serie de casos del problema.

La práctica se evalúa sobre un total de **2 puntos**, distribuidos de la siguiente forma:

- BL (**0.75 puntos**).
- Random **(0.25 puntos).**
- Greedy (**0.5 puntos**).

La fecha límite de entrega será el **el domingo 29 de marzo de 2026** antes de las 23:55 horas. La entrega de la práctica se realizará por internet a través del espacio de la asignatura en PRADO.

2 TRABAJO A REALIZAR

El estudiante deberá de desarrollar los distintos algoritmos al problema planteado. Los métodos desarrollados serán ejecutados sobre una serie de casos del problema. Se realizará un estudio comparativo de los resultados obtenidos y se analizará el comportamiento de cada algoritmo en base a dichos resultados. **Este análisis influirá decisivamente en la calificación final de la práctica**.

En las secciones siguientes se describen los aspectos relacionados con cada algoritmo a desarrollar y las tablas de resultados a obtener.

3 PROBLEMA Y CASOS CONSIDERADOS

3.1 Introducción al Problema del Agrupamiento con Restricciones

El PAR es una generalización del problema del agrupamiento clásico. Permite incorporar al proceso de agrupamiento un nuevo tipo de información: las restricciones. Dado un conjunto de datos X con n instancias, el problema consiste en encontrar una partición $C = \{c_1, c_2, \cdots, c_n\}$ que agrupe dichas instancias en k grupos de forma que se minimice la distancia media entre las instancias del mismo grupo y que se cumplan las restricciones de instancia definidas en el conjunto R al máximo posible. Dichas restricciones implican que existen una serie de parejas de instancias que deben pertenecer (Must-Link, ML) o que no pueden pertenecer (Cannot-Link, CL) al mismo grupo.

El objetivo es obtener una partición solución que permita minimizar la función fitness definida como

$$fitness(solucion) = Desviacion\,(solucion) + \lambda \cdot infeasibility(solucion) \tag{1}$$

. Como se puede observar, es un problema con dos objetivos que se integran en una función mono-objetivo mediante una suma ponderada con el parámetro $\lambda \in [0, 1)$, tal y como se describe en el Seminario 2. Para asegurar que el factor tiene la suficiente relevancia establecemos como el cociente entre la distancia máxima existente en el conjunto de datos y el número de restricciones presentes en el problema, R: $lambda = \frac{max_{dist}(X)}{R}$.

A continuación definimos cómo calcular tanto la distancia *intra-cluster* como la *infeasibility*. En primer lugar, dada una partición C, se puede calcular el centroide asociado a cada grupo c_i como el vector promedio de las instancias de X que lo componen, tal y como se ve en Ecuación (2).

$$\overrightarrow{\mu_i} = \frac{1}{|C_i|} \sum_{x_j \in C_i} \overrightarrow{X_j} \tag{2}$$

Definimos la distancia media intra-cluster de un cluster Ci como la media de las distancias de las instancias que lo conforman a su centroide, tal y como se ve en Ecuación (3).

$$distancia_{intra-cluster}(C_i) = \frac{1}{|C_i|} \sum_{\overrightarrow{x_j} \in C_i} \|\overrightarrow{x_j} - \overrightarrow{\mu}_i\| \tag{3}$$

Definimos la desviación general de la partición $C = \{c_1, c_2, \cdots, c_n\}$ como la media de las desviaciones *intra-cluster* de cada grupo o *cluster* según Ecuación (4).

$$desviacion(C) = \frac{1}{k} \sum_{c_i \in C} distancia_{intra-cluster}(C_i) \tag{4}$$

Dado que disponemos de estructuras para almacenar las restricciones (una matriz de restricciones *MR* o una lista de restricciones *LR*), se puede calcular el valor de *infeasibility* de manera sencilla. Definimos primero como la función que, dada una pareja de instancias, devuelve 1 si la asignación de dichas instancias viola una restricción y 0 en otro caso. A partir de ella, calculamos *infeasibility* como el número de restricciones incumplidas por una partición C del conjunto de datos X de acuerdo a la Ecuación (5).

$$infeasibility = \sum_{i=0}^{n} \sum_{j=0}^{n} V(\overrightarrow{X_i}, \overrightarrow{X_j}) \tag{5}$$

Sin embargo, dado que el porcentaje de restricciones puede ser mucho menor que el total de pares de instancias, en la mayoría de algoritmos (a excepción del algoritmo Greedy) es conveniente calcularlo de forma más directa usando dos vectores de pares de elementos, el de restricciones de grupos distintos ML y el de restricciones del mismo grupo CL:

$$infeasibility(C) = \sum_{i=o}^{|ML|} bool2entero(h_c(\overrightarrow{ML_{i,1}}) \neq h_c(\overrightarrow{ML_{i,2}}) + \sum_{i=o}^{|CL|} bool2entero(h_c(\overrightarrow{CL_{i,1}}) = h_c(\overrightarrow{CL_{i,2}}) \tag{6}$$

donde *bool2entero* es la función indicadora, que devuelve 1 en caso de ser cierta la expresión Booleana que toma como argumento y 0 en otro caso, y ML y CL son los conjuntos de restricciones *Must-Link* y *Cannot-Link*.

3.2 Conjuntos de Datos Considerados

El estudiante trabajará con **6 instancias** del PAR generadas a partir de los **3 conjuntos de datos** siguientes (obtenidos de la web `http://www.ics.uci.edu/~mlearn/MLRepository.html`, procesados en algún caso y disponibles en el espacio de PRADO):

1. Zoo: En donde hay que agrupar animales en función de 16 atributos (si tiene cola, ...). Tiene 7 clases (k = 7) y 101 instancias.
2. Glass: En donde hay que agrupar distintos vidrios en función de 9 atributos sobre sus componentes químicos. Tiene 7 clases (k = 7) y 214 instancias.
3. Bupa: En donde hay que agrupar personas en función de sus hábitos de consumo de alcohol, definido con 5 atributos. Tiene 16 clases (k = 16) y 345 instancias.

A partir de cada conjunto de datos se han obtenido 2 instancias distintas generando 2 conjuntos de restricciones, correspondientes al 15 % y 30 % del total de restricciones posibles.

Los ficheros .dat corresponden a los conjuntos de datos. Los ficheros .const corresponden a los conjuntos de restricciones. Por ejemplo: zoo_set.dat es el fichero de datos de zoo, y los conjuntos de restricciones son zoo_const_set_15.const y zoo_set_const_30.const. Los ficheros de datos contienen en cada fila una instancia del conjunto en cuestión con sus características separadas por comas (sin espacios). Los ficheros de restricciones almacenan una matriz de restricciones con el formato descrito en las transparencias del Seminario 2, con los valores separados por comas y sin espacios.

3.3 Determinación de la Calidad de un Algoritmo

El modo habitual de determinar la calidad de un algoritmo de resolución aproximada de problemas de optimización es ejecutarlo sobre un conjunto determinado de instancias y comparar los resultados obtenidos con los mejores valores conocidos para dichas instancias, si fuesen conocidos.

Además, los algoritmos pueden tener diversos parámetros o pueden emplear diversas estrategias. Para determinar qué valor es el más adecuado para un parámetro o saber la estrategia más efectiva los algoritmos también se comparan entre sí.

La comparación de los algoritmos se lleva a cabo fundamentalmente usando dos criterios, la *calidad* de las soluciones obtenidas y el *tiempo de ejecución* empleado para conseguirlas. Además, es posible que los algoritmos no se comporten de la misma forma si se ejecutan sobre un conjunto de instancias u otro.

Por otro lado, a diferencia de los algoritmos determinísticos, los algoritmos probabilísticos se caracterizan por la toma de decisiones aleatorias a lo largo de su ejecución. Este hecho implica que un mismo algoritmo probabilístico aplicado al mismo caso de un problema pueda comportarse de forma diferente y por tanto proporcionar resultados distintos en cada ejecución.

Cuando se analiza el comportamiento de un algoritmo probabilístico en un caso de un problema, se desearía que el resultado obtenido no estuviera sesgado por una secuencia aleatoria concreta que pueda influir positiva o negativamente en las decisiones tomadas durante su ejecución. Por tanto, resulta necesario efectuar varias ejecuciones con distintas secuencias probabilísticas y calcular el resultado medio (y a veces la desviación típica) de todas las ejecuciones para representar con mayor fidelidad su comportamiento.

Dada la influencia de la aleatoriedad en el proceso, es recomendable disponer de un generador de secuencia pseudoaleatoria de buena calidad con el que, dado un valor semilla de inicialización, se obtengan números en una secuencia lo suficientemente grande (es decir, que no se repitan los números en un margen razonable) como para considerarse aleatoria. En el espacio de PRADO se puede encontrar una implementación en lenguaje C++ de un generador aleatorio de buena calidad (*random.hpp*).

Como norma general, el proceso a seguir consiste en realizar un número de ejecuciones diferentes de cada algoritmo probabilístico considerado para cada caso del problema. Es necesario asegurarse de que se realizan diferentes secuencias aleatorias en dichas ejecuciones. Así, el valor de la semilla que determina la inicialización de cada secuencia deberá ser distinto en cada ejecución y estas semillas deben mantenerse en los distintos algoritmos (es decir, la semilla para la primera ejecución de todos los algoritmos debe ser la misma, la de la segunda también debe ser la misma y distinta de la anterior, etc.). Por simplificar y facilitar la reproducibilidad, se usará la misma semilla para todos los casos de uso. Para mostrar los resultados obtenidos con cada algoritmo en el que se hayan realizado varias ejecuciones, se suelen construir tablas que recojan los valores correspondientes a estadísticos como el **mejor** y **peor** resultado para cada caso del problema, así como la **media** y la **desviación típica** de las ejecuciones. También se pueden emplear descripciones más representativas como los boxplots, que proporcionan información de todas las ejecuciones realizadas mostrando mínimo, máximo, mediana y primer y tercer cuartil de forma gráfica. Finalmente, se suelen construir unas tablas globales con los resultados agregados que mostrarán la calidad del algoritmo en la resolución del problema desde un punto de vista general.

En el PAR consideraremos 50 ejecuciones con semillas de generación de números aleatorios diferentes para cada algoritmo en cada conjunto de datos con su conjunto de restricciones. Esto quiere decir que un algoritmo cualquiera, se ejecutará 50 veces por cada conjunto de datos (con semillas distintas,

y las mismas para todos los algoritmos), por lo que supondrá un total de 50×6=300 ejecuciones por algoritmo.

El resultado final de las medidas de validación será calculado como la media de los 50 valores obtenidos para cada conjunto de datos y restricciones.

Para facilitar la comparación de algoritmos en las prácticas del PAR se considerarán cuatro estadísticos distintos denominados *Fitness*, Tasa_inf, Tasa_C y Tiempo:

Fitness corresponde al valor de la función objetivo que está optimizando el algoritmo; es decir, al valor a minimizar proveniente de la fórmula de la Ecuación 1. Igualmente, se indicará el valor medio de las distintas ejecuciones. Es el criterio usado por cada uno de los algoritmos para optimizar.

Distancia se calcula como el valor promedio del de la distancia *intra-cluster* obtenida. Usamos esta medida ya que si se incumplen las restricciones es posible obtener una distancia *intra-cluster* mejor que si se cumpliesen, así que medimos cómo de reducida es la distancia *intra-cluster*.

Incumplimientos se refiere al promedio de reglas que se han incumplido, entre 0 y 1. Usamos esta medida, al igual que la anterior, para valorar los incumplimientos que obtiene la solución obtenida por el algoritmo.

Evaluaciones indica el total de evaluaciones realizadas por el algoritmo. Debería de ser un valor entre 1 (que debería de tener el Greedy) y 100000. Puede ser menos si el algoritmo para antes de tiempo (por haber encontrado óptimo local).

Tiempo se calcula como la media del tiempo de ejecución empleado por el algoritmo para resolver cada caso del problema (cada conjunto de datos con su conjunto de restricciones). Es decir, en nuestro caso es la media del tiempo empleado en las 50 ejecuciones. Se dará el valor medido en segundos.

En principio cuanto menor es el valor de *Fitness* mejor guía el algoritmo la búsqueda. De todos modos, para valorar cómo de bueno es el algoritmo para el problema es necesario valorar la calidad de la solución final obtenida. Para ello, miraremos primordialmente *Incumplimientos*, ya que cuanto menor sea obtiene agrupamientos más respetuosos con las restricciones. Del mismo modo, cuanto menor es el valor de *Distancia*, mejor calidad tiene también el algoritmo, porque genera agrupamientos más parecidos al ideal. Si dos métodos obtienen soluciones con la misma calidad (tienen valores de *Incumplimientos* y *Distancia* similares), uno será mejor que el otro si emplea menos tiempo en media. En la web de la asignatura hay también disponible un código en C (timer) para un cálculo adecuado del tiempo de ejecución de los algoritmos meta-heurísticos.

En la sección 5 se puede consultar cómo se deben de mostrar los resultados.

4 ALGORITMOS A COMPARAR

4.1 Algoritmo Aleatorio

El algoritmo *aleatorio* se basa en generar **100000** soluciones de forma totalmente aleatoria, y devolver la mejor solución encontrada.

4.2 Algoritmo *Greedy COPKM*

El algoritmo escogido para ser comparado con las metaheurísticas implementadas en esta práctica (la BL) y en las siguientes es el greedy COPKM, que también deberá ser implementado por el estudiante. El objetivo de este algoritmo será generar un agrupamiento basado en el famoso algoritmo k-medias pero teniendo en cuenta la satisfacción de restricciones asociadas al conjunto de datos.

El algoritmo COPKM hace también una interpretación débil de las restricciones. El algoritmo k-medias clásico minimiza únicamente la desviación general. Al modificarlo para tener en cuenta las restricciones, **el algoritmo acomete un problema más complejo y puede tardar más en encontrar una solución**.

A continuación, se describe el algoritmo en los siguientes pasos:

- Se generan k centroides iniciales de forma aleatoria dentro del dominio asociado a cada dimensión del conjunto de datos.
- Se barajan los índices de las instancias para recorrerlas de forma aleatoria sin repetición.
- Se asigna cada instancia del conjunto de datos al grupo más cercano (aquel con centroide más cercano) que viole el menor número de restricciones ML y CL.
- Se actualizan los centroides de cada grupo de acuerdo al promedio de los valores de las instancias asociadas a su grupo.
- Se repiten los dos pasos anteriores mientras que al menos un grupo haya sufrido algún cambio.

El algoritmo COPKM deberá ser ejecutado 50 veces para cada conjunto de datos.

4.3 Algoritmo de Búsqueda Local (BL)

Como algoritmo de BL para el PAR consideraremos el esquema del primer mejor, tal y como está descrito en las transparencias del Seminario 2.

El algoritmo de BL tiene las siguientes componentes, varias de las cuales serán comunes a los algoritmos metaheurísticos de las siguientes prácticas:

Esquema de representación Se seguirá la representación entera basada en un vector S de tamaño n (número de instancias del conjunto de datos) con valores en $\{1, \cdots, k\}$ (siendo k el número de agrupamientos, que coincidirá con el del clases del conjunto de datos real) que indican el cluster asociado a cada instancia, explicado en las transparencias del seminario.

Función objetivo Será la expresión ya explicada en la Ecuación 1. El valor de λ está explicado en las transparencias del Seminario 2.

Generación de la solución inicial La solución inicial se generará de forma aleatoria escogiendo un valor en $\{1, \cdots, k\}$ en cada posición del vector S, comprobando que cada cluster tiene al menos una instancia asignada.

Esquema de generación de vecinos Se emplea el movimiento de cambio de cluster que altera el vector S sustituyendo el valor si en una posición i-ésima por otro valor $l \in \{1, \cdots, k\}$ y $l \neq s_i$, comprobando que no deja ningún cluster vacío.

Criterio de aceptación Se considera una mejora cuando disminuye el valor global de la función objetivo.

Exploración del entorno Se realizará una exploración en orden aleatorio del entorno en cada iteración.

Se detendrá la ejecución del algoritmo de BL cuando no se encuentre mejora en todo el entorno o cuando se hayan realizado 100000 evaluaciones de la función objetivo, es decir, en cuanto se cumpla alguna de las dos condiciones.

5 TABLAS DE RESULTADOS A OBTENER

Se mostrará una tabla global por cada problema y nivel de restricciones, en el que mostrará para cada algoritmo (COPKM, Random, BL) los resultados de la ejecución de dicho algoritmo en los conjuntos de datos considerados. Para rellenar esta tabla se hará uso de los resultados medios mostrados en las tablas parciales. Aunque en la tabla que sirve de ejemplo se han incluido todos los algoritmos considerados en esta práctica, naturalmente sólo se incluirán los que se hayan desarrollado. Por tanto, se tendrán que hacer 6 tablas, que tendrán la misma estructura que las Tablas 1.

Tabla 1: Resultados obtenidos para el dataset XX con XX % de restricciones

Algoritmo	Fitness	Distancia	Incumplimiento	Evaluaciones	Tiempo (seg)
Greedy	x	x	x	1	x
Random	x	x	x	x	x
BL	x	x	x	x	x

Adicionalmente, se deberá de realizar para cada ejemplo una visualización en boxplot mostrando la distribución de datos de cada algoritmo sobre la medida de fitness. Un ejemplo sería la gráfica de la Figura 1.

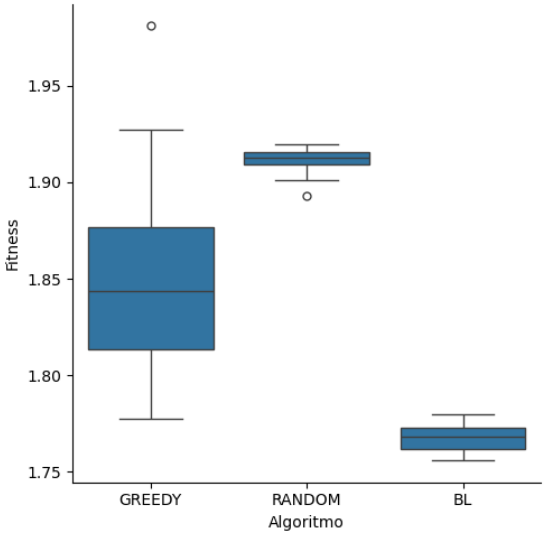

Figura 1: Ejemplo de Boxplot con los 50 valores por algoritmo

A partir de los datos mostrados en estas tablas, el estudiante realizará un análisis de los resultados obtenidos, **que influirá significativamente en la calificación de la práctica**. En dicho análisis se deben comparar los distintos algoritmos en términos de calidad de las soluciones y tiempo requerido para producirlas. Por otro lado, se puede analizar también el comportamiento de los algoritmos en algunos de los casos individuales que presenten un comportamiento más destacado.

6 DOCUMENTACIÓN Y FICHEROS A ENTREGAR

En general, la **documentación** de ésta y de cualquier otra práctica será un fichero pdf que deberá incluir, al menos, el siguiente contenido:

a) Portada con el número y título de la práctica (con el nombre del problema), el curso académico, el nombre, DNI y dirección e-mail del estudiante, y su horario de prácticas.

b) Índice del contenido de la documentación con la numeración de las páginas.

c) **Breve** descripción/formulación del problema (**máximo 1 página**). Podrá incluirse el mismo contenido repetido en todas las prácticas presentadas por el estudiante.

d) Breve descripción de la aplicación de los algoritmos empleados al problema (**máximo 4 páginas**): Todas las consideraciones comunes a los distintos algoritmos se describirán en este apartado, que será previo a la descripción de los algoritmos específicos. Incluirá por ejemplo la descripción del esquema de representación de soluciones y la **descripción en pseudocódigo (no código)** de la función objetivo y los operadores comunes.

e) Descripción en **pseudocódigo** de la **estructura del método de búsqueda** y de todas aquellas **operaciones relevantes** de cada algoritmo. Este contenido, específico a cada algoritmo se detallará en los correspondientes guiones de prácticas. El pseudocódigo **deberá forzosamente reflejar la implementación/ el desarrollo realizados** y no ser una descripción genérica extraída de las transparencias de clase o de cualquier otra fuente. La descripción de cada algoritmo no deberá ocupar más de **2 páginas**.

   Para esta primera práctica, se incluirán al menos las descripciones en pseudocódigo de:

   - Para el algoritmo *greedy*, aparte del esquema general, la función heurística.
   - Para el algoritmo BL, el métodos de exploración del entorno, el operador de generación de vecino, la factorización de la BL si fuese el caso, y la generación de soluciones aleatorias.

f) Breve explicación de la **estructura del código de la práctica**, incluyendo un pequeño **manual de usuario describiendo el proceso para que el profesor de prácticas pueda compilarlo (usando un sistema automático como make o similar) y cómo ejecutarlo, dando algún ejemplo de ejecución**.

g) Experimentos y análisis de resultados:

   a) Descripción de los casos del problema empleados y de los valores de los parámetros considerados en las ejecuciones de cada algoritmo (**incluyendo las semillas utilizadas**).

   b) Resultados obtenidos según el formato especificado.

   c) Análisis de resultados (**Máximo: 4 páginas sin contar las figuras**). El análisis deberá estar orientado a **justificar** (según el comportamiento de cada algoritmo) **los resultados** obtenidos en lugar de realizar una mera "lectura" de las tablas. Se valorará la inclusión de otros elementos de comparación tales como gráficas de convergencia, boxplots, análisis comparativo de las soluciones obtenidas, representación gráfica de las soluciones, etc.

h) Referencias bibliográficas u otro tipo de material distinto del proporcionado en la asignatura que se haya consultado para realizar la práctica (en caso de haberlo hecho).

Aunque lo esencial es el contenido, también debe cuidarse la presentación y la redacción. **La documentación nunca deberá incluir listado total o parcial del código fuente**.

En lo referente al **desarrollo de la práctica**, se entregará una carpeta llamada **software** que contenga una versión ejecutable de los programas desarrollados, así como el código fuente implementado o los ficheros de configuración del framework empleado. El código fuente o los ficheros de configuración se organizarán en la estructura de directorios que sea necesaria y deberán colgar del directorio *src* en el raíz. Junto con el código fuente, hay que incluir los ficheros necesarios para construir los ejecutables según el entorno de desarrollo empleado (tales como *.prj, makefile, *.ide, etc.). En este directorio se adjuntará también un pequeño fichero de texto de nombre LEEME que contendrá breves reseñas sobre cada fichero incluido en el directorio. Es importante que los programas realizados puedan leer los valores de los parámetros de los algoritmos desde fichero, es decir, que no tengan que ser recompilados para cambiar éstos ante una nueva ejecución. Por ejemplo, la semilla que inicializa la secuencia pseudoaleatoria debería poder especificarse como un parámetro más.

En el caso de que en el lenguaje de programación elegido se haya ofrecido un API, deberá de **obligatoriamente implementarlo siguiendo el API ofrecido**. Si no fuese el caso, se adaptará el API recomendado.

El fichero pdf de la documentación y la carpeta software serán comprimidos en un fichero .zip etiquetado con los apellidos y nombre del estudiante (Ej. Pérez Pérez Manuel.zip). Este fichero será entregado por internet a través del espacio de la asignatura en PRADO.

7 MÉTODO DE EVALUACIÓN

Al principio de la práctica se ha indicado la puntuación máxima que se puede obtener por cada algoritmo y su análisis. **La inclusión de trabajo voluntario** (desarrollo de variantes adicionales, experimentación con diferentes parámetros, prueba con otros operadores o versiones adicionales del algoritmo, análisis extendido, etc.) **podrá incrementar la nota final** por encima de la puntuación máxima definida inicialmente, o compensar parcialmente errores en la práctica.

En caso de que el comportamiento del algoritmo en la versión implementada/ desarrollada no coincida con la descripción en pseudocódigo o no incorpore las componentes requeridas, se podría reducir hasta en un 50 % la calificación del algoritmo correspondiente.